Algoritmos de Busca Informada

(C)Russell & Norvig

Material

- Capítulo 4 - Rusell & Norvig

Roteiro

- Busca pela Melhor Escolha (Best-first)
- Busca Gulosa pela Melhor Escolha (Greedy best-first)
- Busca A^* (minimizar custo total estimado da solução)
- Busca Heurística
- Algoritmos de Busca Local
- Busca de subida de encosta (Hill-climbing)
- Busca de Têmpera Simulada
- Busca em Feixe Local (local beam)
- Algoritmos Genéticos

Lembrando: Busca em árvore

Uma estratégia de busca é definida escolhendo a ordem de expansão dos nós

Busca pela Melhor Escolha (Best-first)

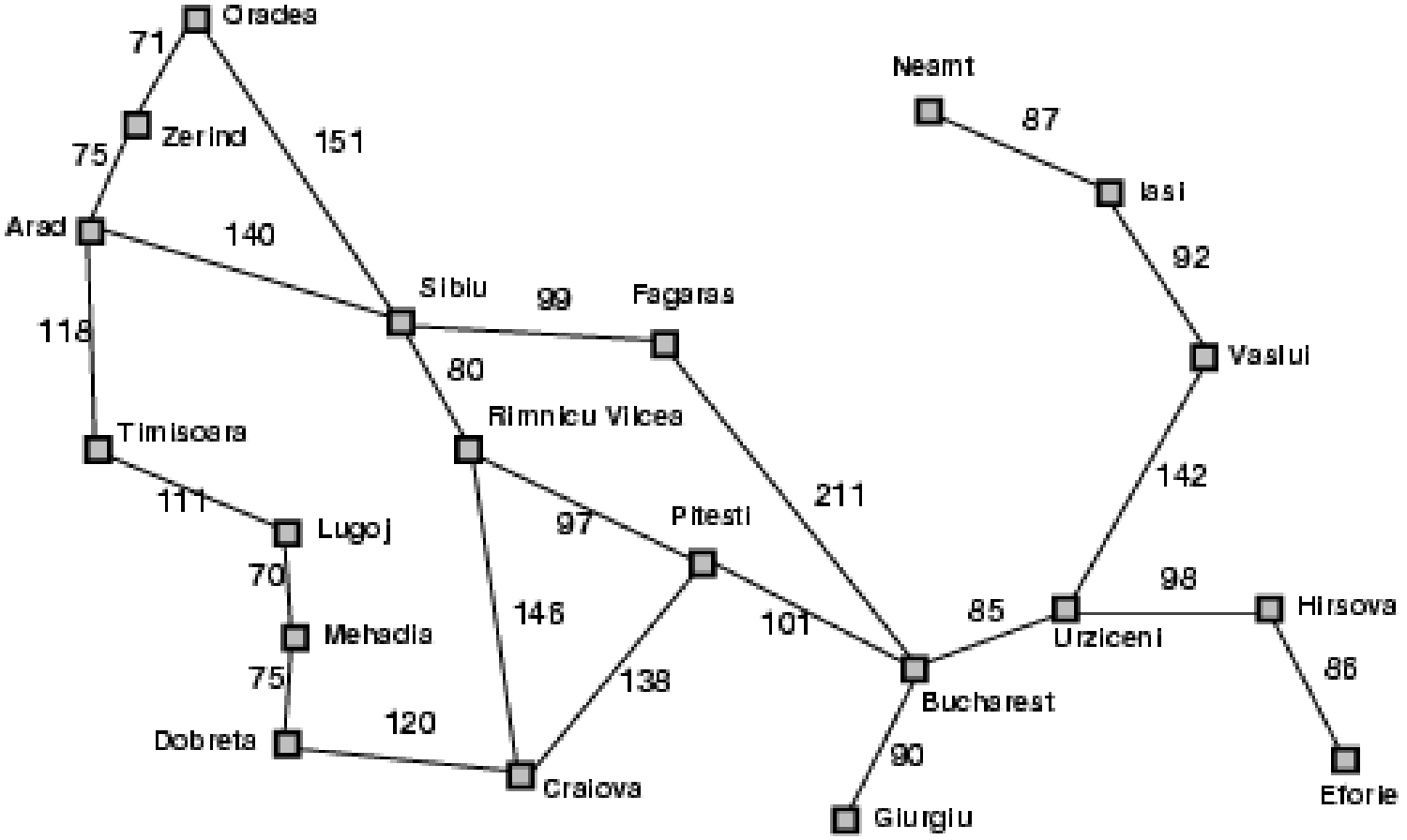
- Idéia: usar uma função de avaliação *f(n) para cada nó*
  - estimar o grau em que um nó é ”desejável” como caminho
  - → Expandir os nós não expandidos mais desejáveis

- Implementação:

  Ordenar os nós na borda em ordem decrescente em função de serem “desejáveis”

- Casos especiais:
  - Busca Gulosa pelo Melhor Caminho
  - Busca A*

Romênia com o custo dos passos em km



Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

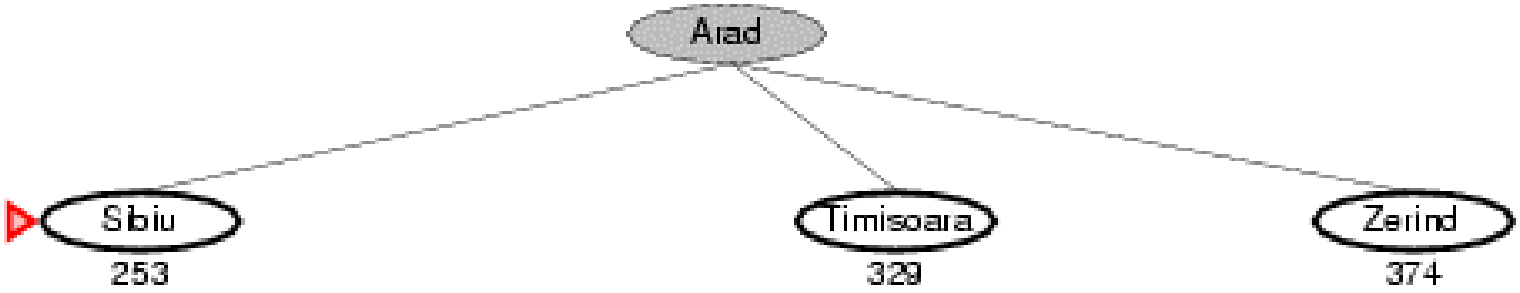
Busca Gulosa pela Melhor Escolha

- Função de Avaliação $f(n) = h(n)$ (heurística)
- O mesmo que estimar o custo para ir de *n* até o *objetivo*
- e.g., $h_{DLR}(n)$ = distância em linha reta de *n* até Bucareste
- Expande o nó que parece ser o mais próximo do objetivo

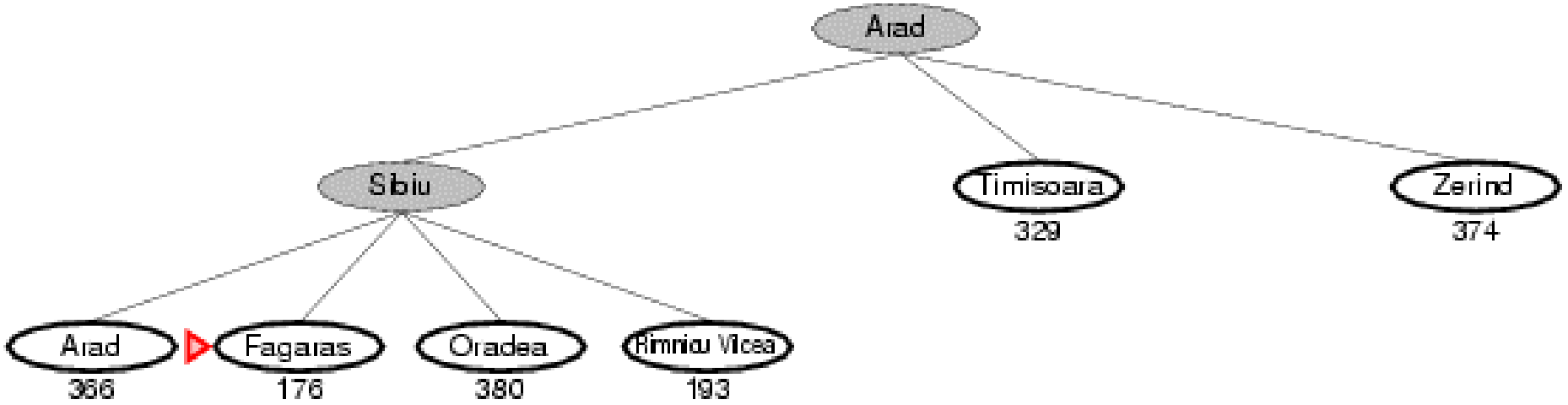
Exemplo de Busca Gulosa pela Melhor Escolha



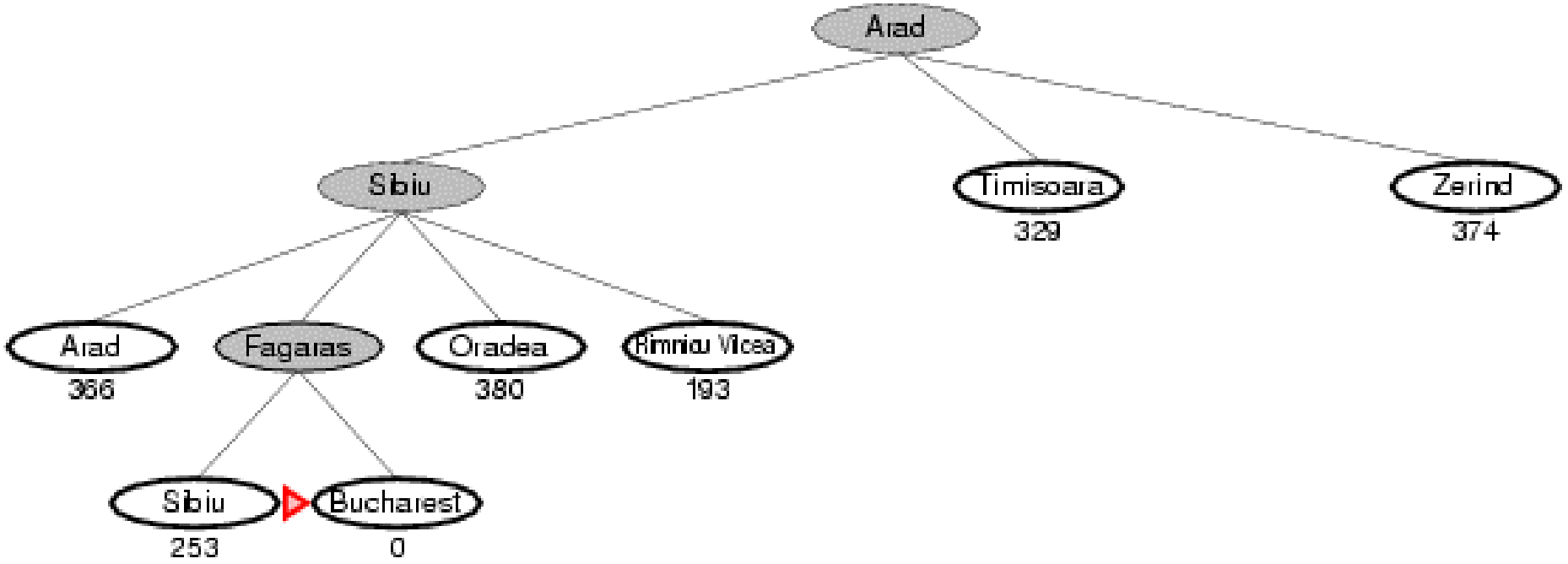
Exemplo de Busca Gulosa pela Melhor Escolha



Exemplo de Busca Gulosa pela Melhor Escolha



Exemplo de Busca Gulosa pela Melhor Escolha



Propriedades da Exemplo de Busca Gulosa pela Melhor Escolha

- Completa? Não – pode ficar preso em laços, e.g., Iasi → Neamt → Iasi → Neamt →
- Tempo? $O(b^m)$, mas uma boa heurística pode fornecer melhoras enormes
- Espaço? $O(b^m)$ – mantém todos os nós na memória
- Ótima? Não

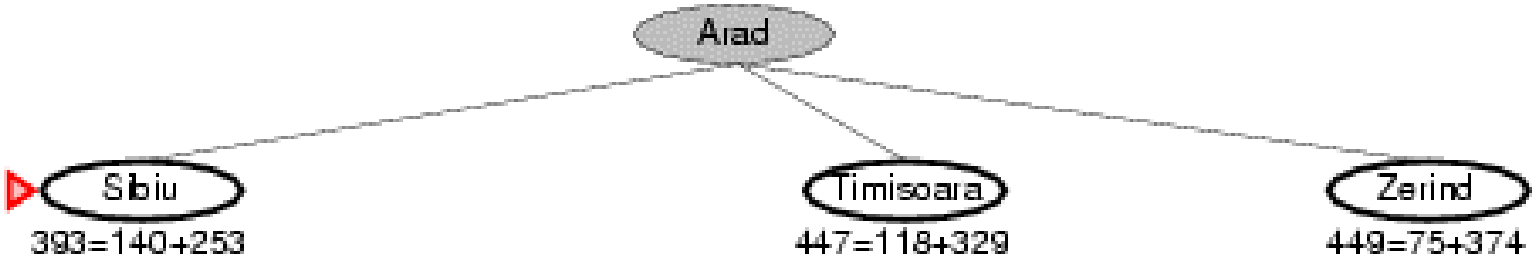
Busca A*

- Idéia: evita de antemão expandir caminhos que são caros
- Função de avaliação $f(n) = g(n) + h(n)$
- $g(n)$ = custo até chegar a n
- $h(n)$ = estimar custo para ir de n até o objetivo
- $f(n)$ = estima o custo total do caminho para chegar ao objetivo passando por n

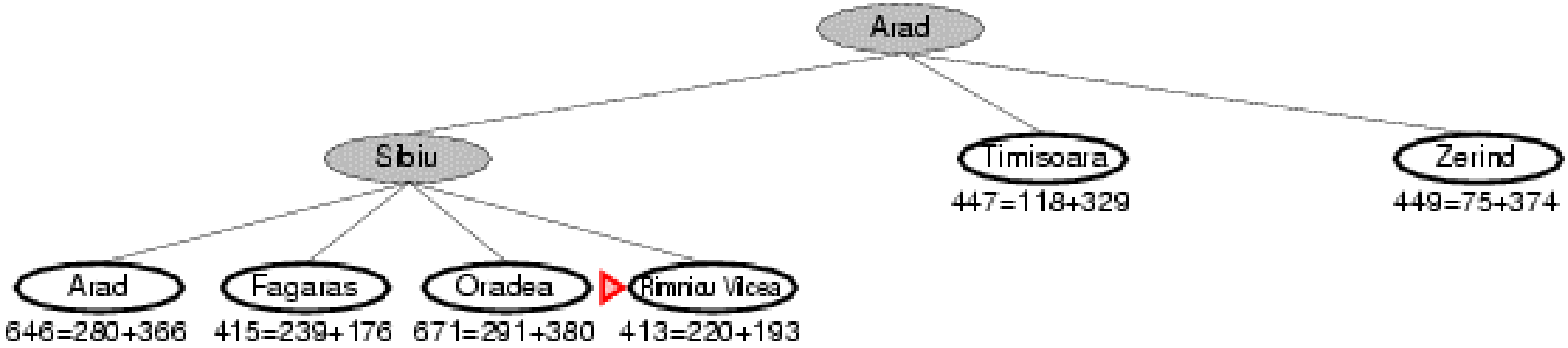
Exemplo de Busca A*



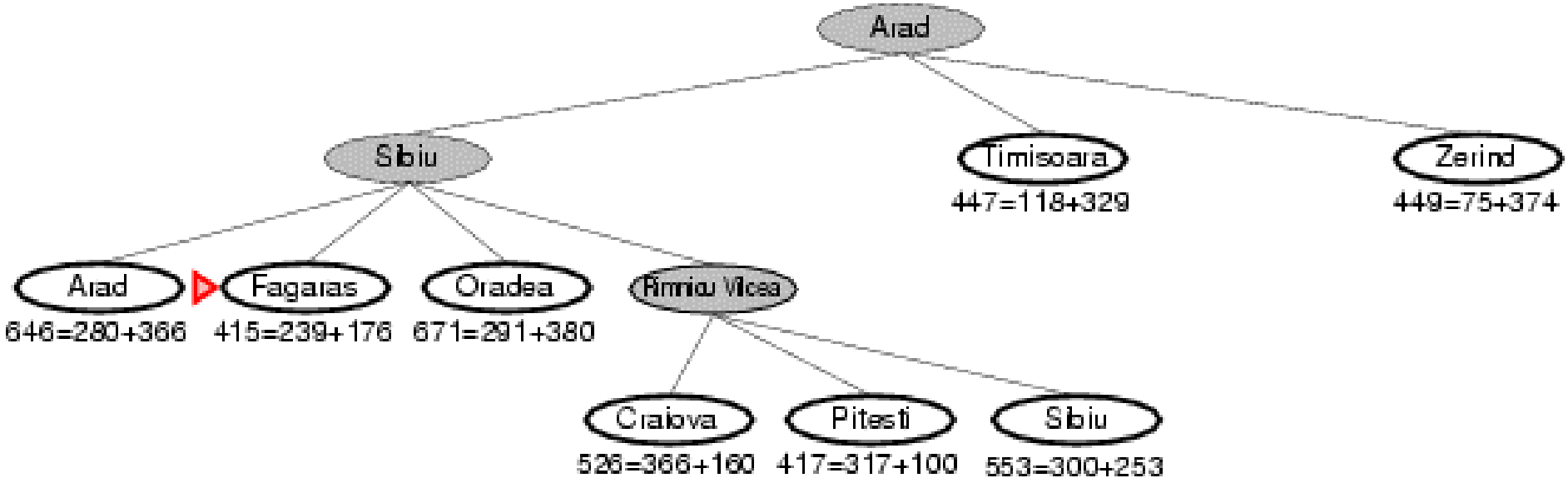
Exemplo de Busca A*



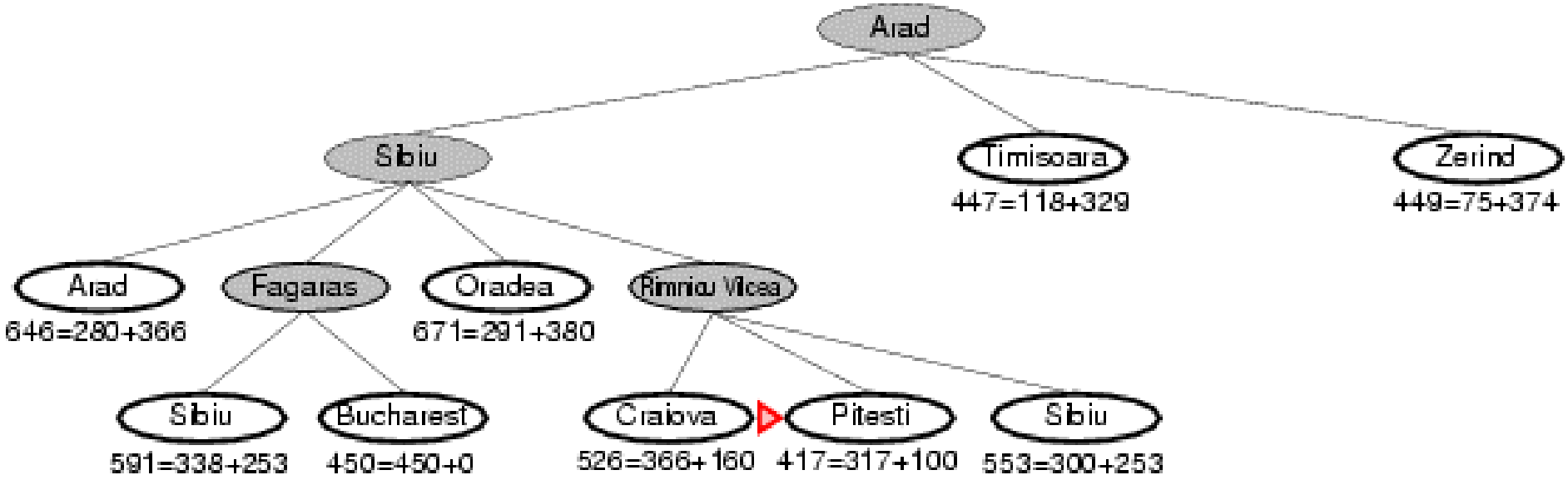
Exemplo de Busca A*



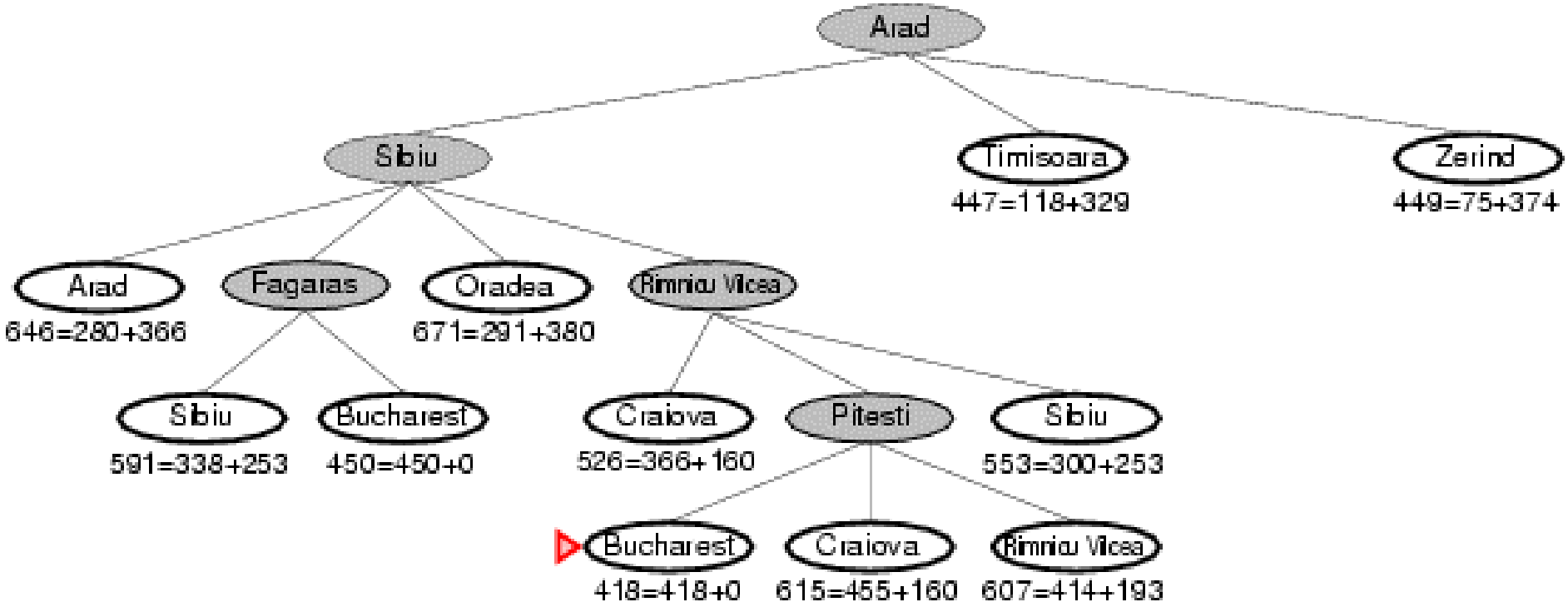
Exemplo de Busca A*



Exemplo de Busca A*



Exemplo de Busca A*



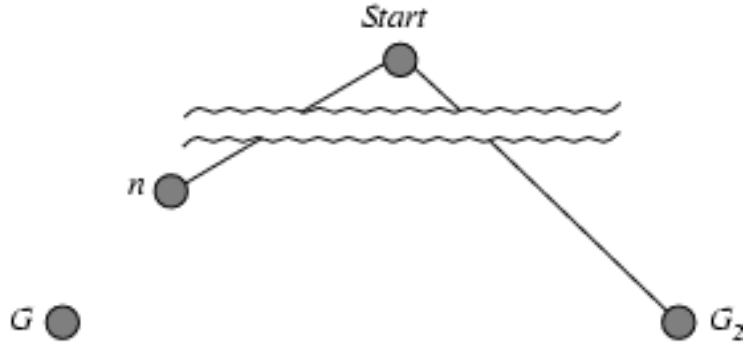
Heurística Admissível

- Uma heurística *h(n)* é admissível se para cada nó *n*,

  $h(n) \leq h^*(n)$, onde $h^*(n)$ é o custo verdadeiro para chegar o estado objetivo a partir de *n*.
- Uma heurística admissível nunca superestima o custo para atingir o objetivo, i.e., é otimista
- Exemplo: $h_{DLR}(n)$ (nunca superestima a distância real na estrada)
- Teorema: Se *h(n) é admissível*, A* usando `BUSCA-EM-ÁRVORE` é ótima

Otimalidade de A* (prova)

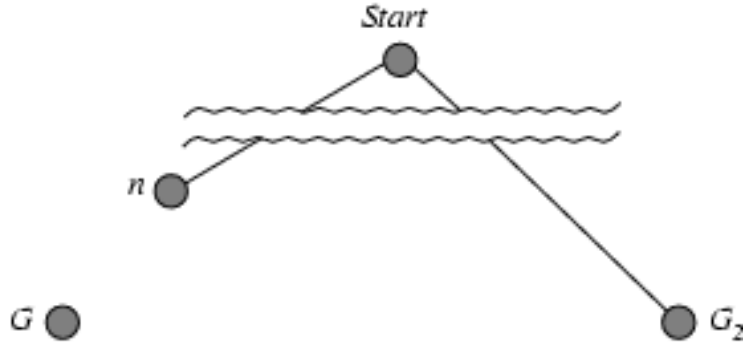
- Suponha que algum objetivo sub-ótimo G_2 foi atingido (gerado) e está na borda. Seja *n* um nó não expandido na borda tal que *n está no caminho mais curto para chegar a um objetivo ótimo G*.



- $f(G_2) = g(G_2)$ como $h(G_2) = 0$
- $g(G_2) > g(G)$ como G_2 é sub-ótimo
- $f(G) = g(G)$ como $h(G) = 0$
- $f(G_2) > f(G)$ como conseqüência

Otimalidade de A* (prova)

- Suponha gora que algum objetivo sub-ótimo *G_2 foi gerado (atingido)* e está na borda. Seja *n* um nó não expandido que está na borda, tal que *n está num caminho mais curto para atingir o objetivo ótimo G*.



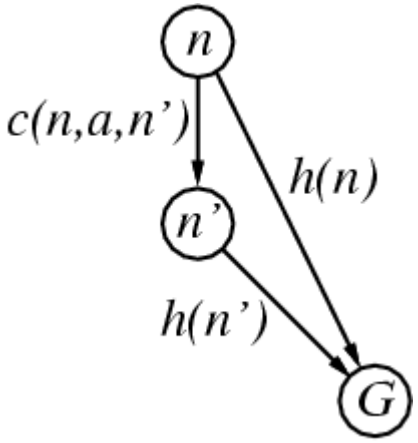
- $f(G_2)$ $> f(G)$ decorre do anterior
- $h(n)$ $\leq h^*(n)$ como h é admissível
- $g(n) + h(n)$ $\leq g(n) + h^*(n)$
- $f(n)$ $\leq$ f(G)˝

Daí *$f(G_2) > f(n)$*, e A* nunca selecionará G_2 para ser expandido

Heurística Consistente (MONOTONICIDADE)

- Uma heurística é consistente se para cada nó *n*, cada sucessor *n'* de *n* gerado por qualquer ação *a*,

  $h(n) \leq c(n,a,n') + h(n')$˝



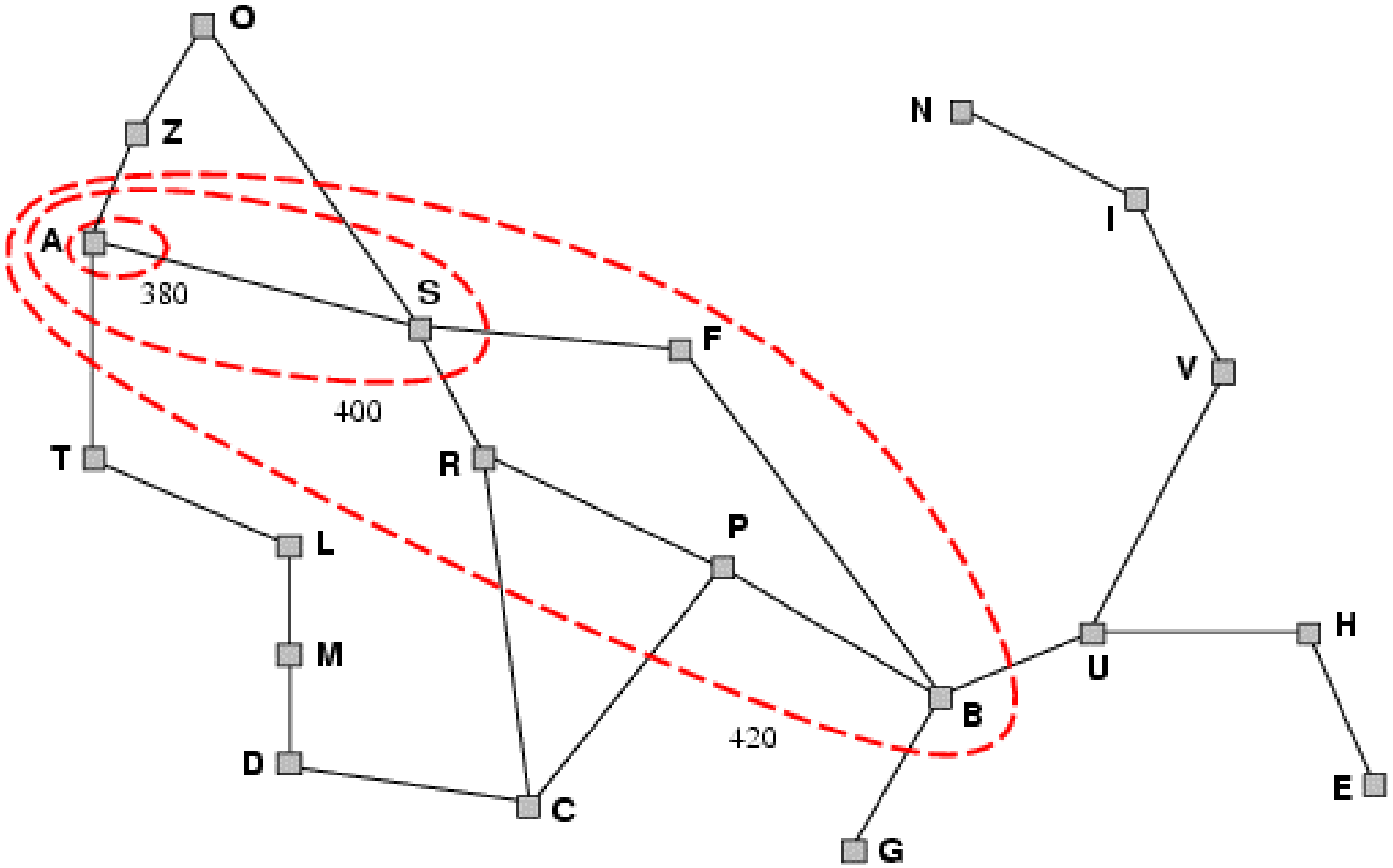
- Se *h* é consistente, nós temos

  f(n') = g(n') + h(n')
  = g(n) + c(n,a,n') + h(n')
  ≥ g(n) + h(n)
  = f(n)

- i.e., *f(n)* é não decrescente ao longo de qualquer caminho.
- Teorema: Se *h(n)* é consistente, A* usando `BUSCA-EM-GRAFO` é ótima

Otimalidade de A*

- A* expande nós em ordem de valores decrescentes de *f*
- Gradualmente vão sendo adicionados os "*f*-contornos" dos nós
- O Contorno *i* possui todos os nós com $f=f_i$, onde $f_i < f_{i+1}$



Propriedades de A*

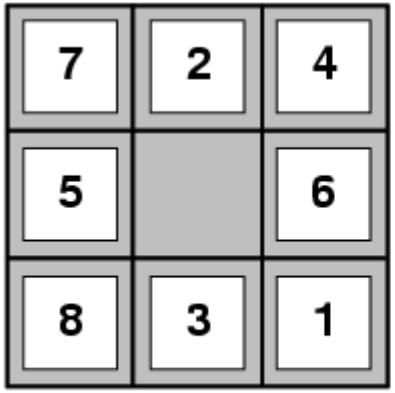
- Completa? Sim (a menos que existam infinitos nós com f $\leq$ *f(G)*)
- Tempo? Exponencial
- Espaço? Guarda todos os nós na memória
- Ótima? Sim

Heurísticas Admissíveis

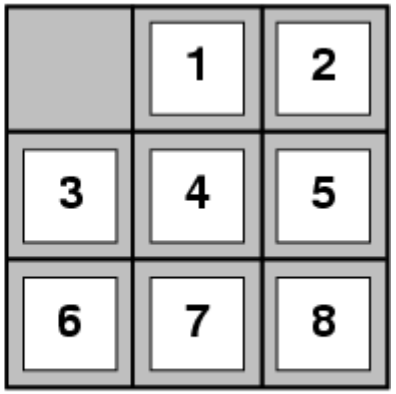
E.g., para o quebra-cabeça de 8 peças

- $h_1(n)$ = número de pedras fora do lugar
- $h_2(n)$ = distância total à la Manhattan

(i.e., número de quadrados da localização desejada de cada pedra)



Start State



Goal State

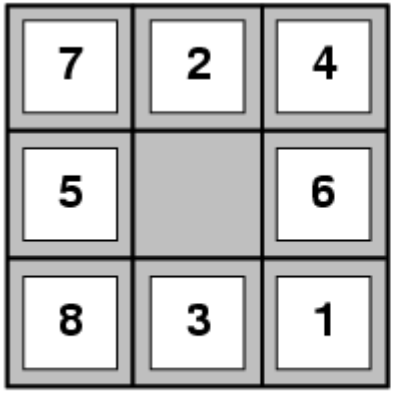
- $h_1(S) = ?$
- $h_2(S) = ?$

Heurísticas Admissíveis

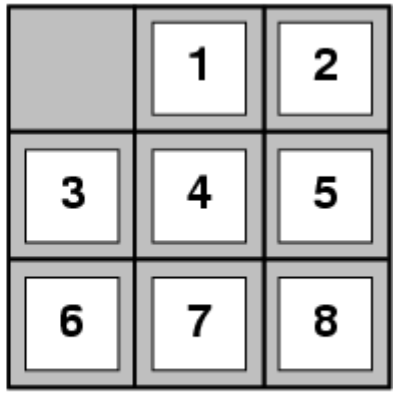
E.g., para o quebra-cabeça de 8 peças

- $h_1(n)$ = número de pedras fora do lugar
- $h_2(n)$ = distância total à la Manhattan

(i.e., número de quadrados da localização desejada de cada pedra)



Start State



Goal State

- $h_1(S) = ?$ 8
- $h_2(S) = ?$ 3+1+2+2+2+3+3+2 = 18

Dominância

- Se $h_2(n) \geq h_1(n)$ para todo n (ambos admissíveis)
- então h_2 domina h_1
- h_2 é melhor para busca

- Custos de busca típicos (número médio de nós expandidos):

- *d=12* *BAI* = 3,644,035 nós
  $A^*(h_1)$ = 227 nós
  $A^*(h_2)$ = 73 nós
- *d=24* *BAI* = nós demais
  $A^*(h_1)$ = 39,135 nós
  $A^*(h_2)$ = 1,641 nós

BAI= Busca por Aprofundamento Iterativo

Problemas com menos restrições

- Um problema com menos restrições nas ações é chamado as vezes relaxado
- O custo de uma solução ótima para um problema relaxado é uma heurística admissível para o problema original
- Se as regras do quebra-cabeças com 8 peças são relaxadas tal que uma pedra pode ser movimentado para qualquer posição, então *$h_1(n)$ fornece a solução mais curta*
- Se as regras são relaxadas para que as pedras possam ir para qualquer quadrado adjacente, então *$h_2(n)$ fornece a solução mais curta*

Algoritmos de Busca Local

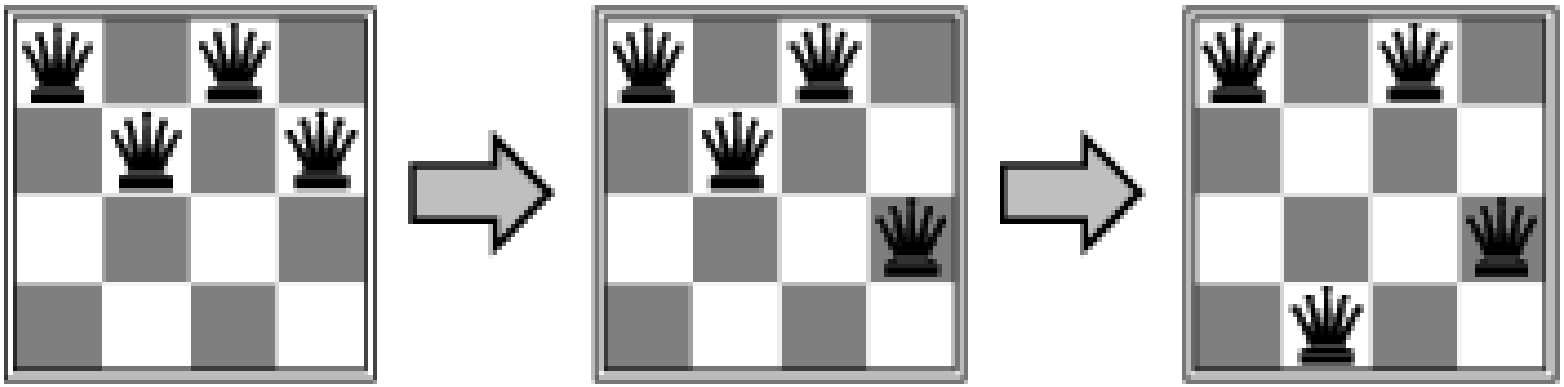
- Em muitos problemas de otimização o caminho ao objetivo é irrelevante, o estado objetivo é a solução

- Espaço de estados = conjunto de configurações “completas”
- Achar configurações que satisfaçam restrições, e.g., N-Rainhas

- Nestes casos, podem ser usados algoritmos de busca local
- Manter um só estado como o “atual" e tentar melhorar o mesmo

Exemplo: *N-rainhas*

- Por *n* rainhas em um tabuleiro $n \times n$ sem que elas atinjam umas as outras



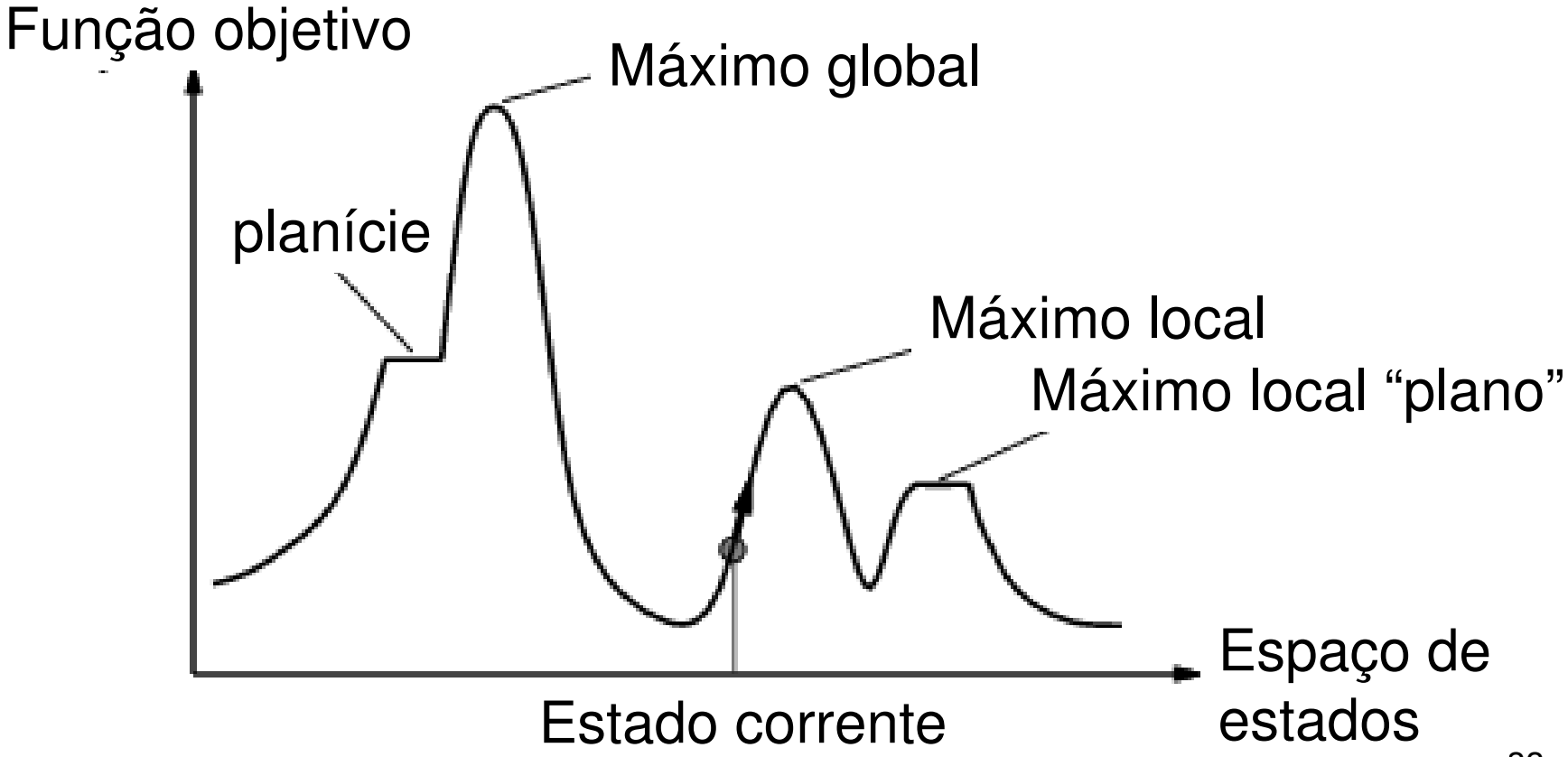
Busca de Subida de Encosta (Hill-climbing)

- "Escalando o Everest no meio de uma tempestade de neve com amnésia"

função SUBIDA-DE-ENCOSTA(*problema*) **retorna** um estado que é o máximo local
entrada: *problema*
variáveis locais: *corrente* /* um nó*/
vizinho /* um nó*/
repita
vizinho ← um sucessor de *corrente* com valor mais alto
se VALOR[*vizinho*] ⩽ VALOR[*corrente*] **então retornar** ESTADO[*corrente*]
corrente ← *vizinho*

Busca de Subida de Encosta

- Problema: dependendo do estado inicial, pode ficar preso num mínimo local



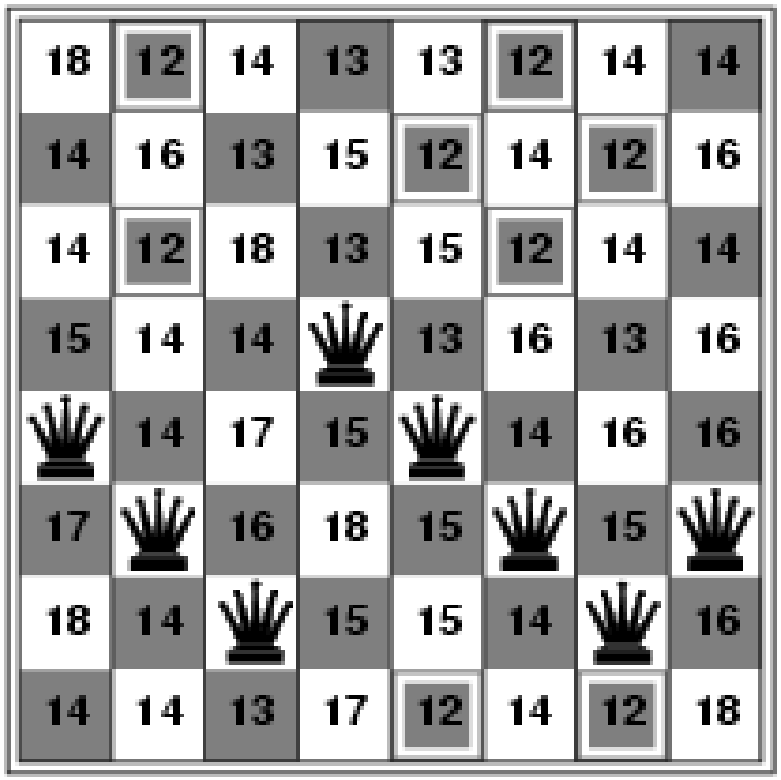
Busca de subida de encosta: problema das 8-rainhas

- *h= custo de heurística*

- *Função sucessora* retorna todos os estados possíveis gerados pela movimentação de uma única rainha para outro quadrado na mesma coluna (de forma que cada quadrado tenha 8x7=56 sucessores)

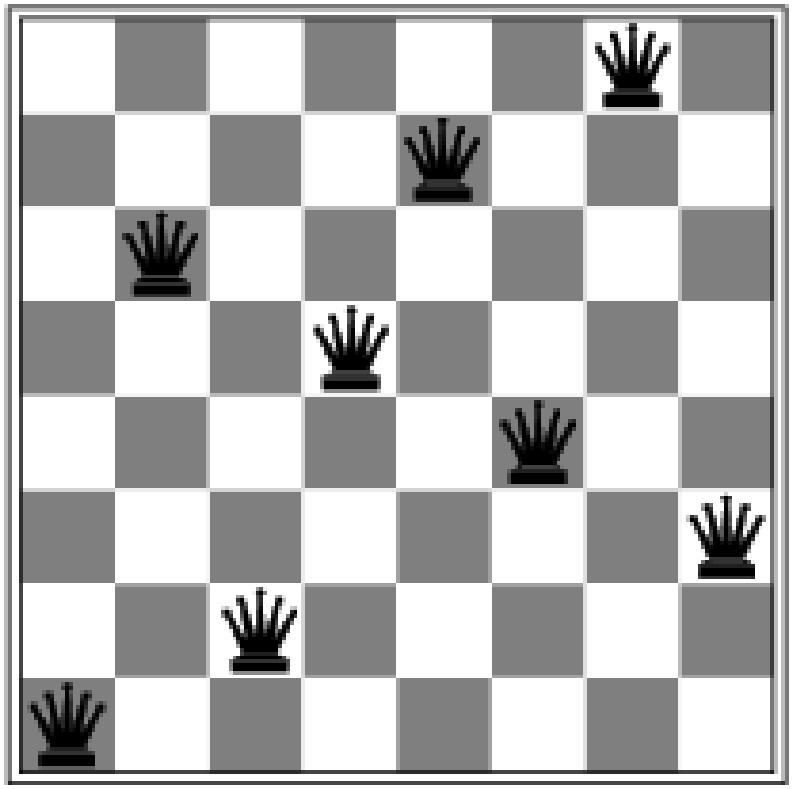
- mínimo global=0 somente para as soluções perfeitas

Busca de subida de encosta: as 8-rainhas



- *h* = número de pares de rainhas que estão atacando umas às outras, direta ou indiretamente
- *h = 17* para o estado mostrado acima

Busca de subida de encosta: problema das 8-rainhas



- Um mínimo local com *h = 1*

Busca de Têmpera Simulada

- Idéia: fugir de máximo local permitindo alguns movimentos ”ruins" mas decrescendo gradualmente a sua freqüência

função TÊMPERA-SIMULADA(*problema, escalonamento*) **retorna** um estado solução
entrada: *problema*
escalonamento /* *um mapeamento de tempo para “temperatura”*/*
variáveis locais: *corrente* /* um nó*/
próximo /* um nó*/
T /* “temperatura” que controla a probabilidade de passos descendentes*/
corrente ← CRIAR-NÓ(ESTADO-INICIAL[problema])
para *t* ← 1 **até** ∞ **faça**
T ← *escalonamento*[*t*]
se *T*=0 **então retornar** *corrente*
próximo ← um *sucessor* de corrente selecionado ao acaso
ΔE ← VALOR[*próximo*] - VALOR[*corrente*]
se ΔE > 0 **então** *corrente* ← *próximo*
senão *corrente* ← *próximo* somente com probabilidade $e^{\Delta E/T}$ /*prob. Diminui exponencialmente com movimento ruim*/

Propriedades da busca de têmpera simulada

- Pode ser provado que:

  Se *T* decresce de maneira suficientemente lenta,a busca por têmpera simulada encontrará um ótimo global com probabilidade próxima de 1(100%)!!

- Muito usado para projetar layout de VLSI, escalonamento de vôos, etc.

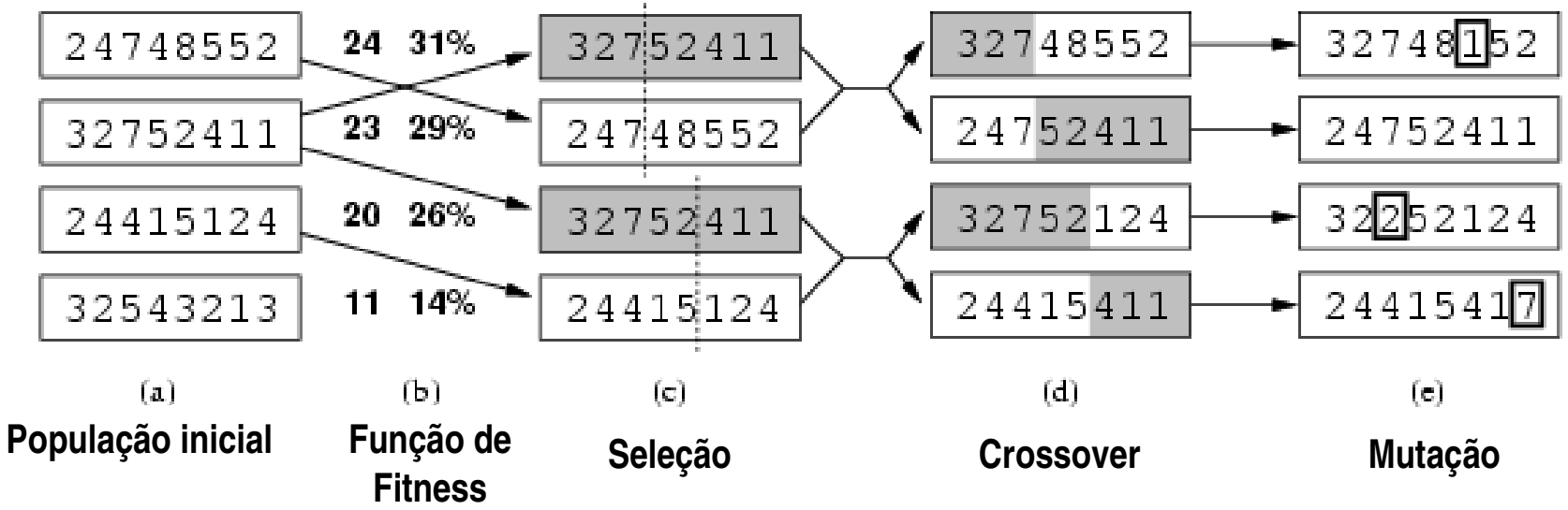
Busca em feixe local

- Monitora *k* estados ao invés de um só
- Começa com *k* estados selecionados aleatoriamente
- Em cada iteração, todos os sucessores de todos os *k* estados são gerados
- Se qualquer um dos estados é o objetivo, para; senão seleciona os *k* melhores sucessores da lista completa e repete

Algoritmos Genéticos

- Um estado sucessor é gerado combinando dois estados "pais"
- Inicia com *k* estados gerados aleatoriamente (população)
- Um estado é representado como uma string sobre um alfabeto finito (em geral uma string de 0s e 1s)
- Função de avaliação (função de fitness). Valores mais altos para estados melhores
- Produção da próxima geração de estados através de seleção, crossover e mutação

Algoritmos Genéticos



- Função de “Fitness” (“ajuste” apropriado): número de pares de rainhas não-atacantes (min = 0, max = 8 × 7/2 = 28)
- 24/(24+23+20+11) = 31%
- 23/(24+23+20+11) = 29% etc

Algoritmos Genéticos

